

DevSecOps 5-1-Intro-to-IaC

Task 1 - Introduction

Infrastructure as Code (IaC) is a foundational element of DevSecOps, and this module provides a structured introduction to its key concepts and tools for newcomers.

Task 2 - IaC - The Concept

Let us consider what was required when infrastructure was configured and managed in the days before IaC. Here are some tasks which would have been the responsibility of various teams (depending on companies structure), such as DevOps/DevSecOps, Platform, Infrastructure, Network and Systems Administration:

- Manually provisioning servers (physically installing servers or manually deploying lab machines)
- Network configuration (setting up a network for the infrastructure, including setting up IP addresses, subnets, routing tables and any network components that the infra needs like routers, switches or firewalls)
- Installing an OS
- Installation, configuration and updating of software (database, web server etc.)
- Disaster recovery (DR) (this involves having a backup infrastructure on standby, making a detailed "failover" plan, and having clear procedures in place should the primary data centre go down).

Infrastructure as Code Explained: Infrastructure as Code (IaC) replaces manual sysadmin workflows by using software blueprints to automatically provision, scale, and maintain infrastructure in a defined desired state. For example, you may define some resources in your code, such as network components, lab machines, and a database. An IaC tool will build this blueprint and ensure the infrastructure is always in line with it. Should you want to make an infrastructure change, like

increasing the number of VMs, only a simple change needs to be made to the code, which will be reflected in your infrastructure once applied.

The Many Benefits of IaC: Infrastructure as Code (IaC) transforms infrastructure into a collaborative, shared asset that integrates directly into CI/CD pipelines for consistent cross-environment deployments. By capturing configurations in versioned code, it eliminates knowledge silos, human error, and manual overhead to deliver highly scalable and repeatable infrastructure.

Scalable: IaC enables instant, cost-effective resource scaling via single commands or automated triggers, replacing weeks of slow manual ticketing.

Versionable: Storing infrastructure in version control creates an automated audit trail and enables instant rollbacks to historical working states during outages.

Repeatable: A single IaC configuration deploys identical environments across dev, staging, and production, eliminating configuration drift and software testing inconsistencies.

IaC in Practice: While manual disaster recovery during data center outages is slow and error-prone, IaC streamlines high-pressure failover processes for critical business continuity. Here are a few ways in which IaC can help with disaster recovery:

- **Infrastructure Automation:** IaC streamlines failover by programmatically provisioning and configuring replacement resources in secondary data centers without manual setup.
- **Data Backup and Recovery:** Codified recovery procedures automatically trigger during outages to restore application data and minimize costly data loss.
- **Failover Automation:** Traffic and critical service routing automatically switch over to backup infrastructure, significantly reducing operational downtime.
- **Configuration Consistency:** Declarative state enforcement eliminates human error and misconfigurations when spinning up secondary environments during crisis recovery.
- **Scaling Resources:** Automated resource scaling dynamically adjusts compute capacity to absorb heavy disaster recovery workloads and scales down afterward to control costs.

- **Operational Impact:** By codifying scalability, versioning, and repeatability, IaC eliminates operational overhead and saves teams significant time during routine and critical operations.

Infrastructure as Code (IaC) is a core DevSecOps capability that accelerates delivery by uniting version-controlled collaboration in CI pipelines with automated, consistent infrastructure deployment across all environments in CD.

Answer the questions below

Your organisation is preparing to launch a new service called FlyNet. The DevSecOps team provisioned an infrastructure and tested this service in the dev environment. Which IaC characteristic will streamline the provisioning of this same infrastructure in staging and production?

Repeatable

It's the day before launch, and the latest infra change has started producing strange errors, something about "destroying humanity". Weird! Which IaC characteristic allows us to go back to the last known working version?

Versionable

It's launch day, and it couldn't have gone better; the service is almost running itself! The FlyNet launch has attracted a lot of new customers! Which IaC characteristic enables us to increase the resources available to our infrastructure to meet this increased demand with ease?

Scalable

Task 3 - IaC - The Tools Part 1

End-to-end IaC deployment relies on combining specialized tools—from multi-cloud provisioners like Terraform to configuration managers like Ansible—tailored to their specific functional strengths across the infrastructure lifecycle.

Declarative vs. Imperative Approaches

Declarative: Declarative IaC defines the target end-state via a tracking state file to ensure idempotency and prevent unnecessary changes upon re-execution.

Imperative: Imperative IaC executes procedural commands step-by-step regardless of current state, offering fine-grained control but risking configuration drift or duplicate provisioning if rerun.

Declarative tools evaluate your current location to determine the path to the desired end-state, whereas imperative tools run a fixed sequence of instructions that require a specific starting point to succeed.

Agent-based vs. Agentless Architecture

Agent-based: Agent-based tools install dedicated background software on target nodes for detailed telemetry and offline execution, though they add software maintenance overhead and expand the network attack surface through open ports.

Running agents on target systems grants granular control and offline execution support, but introduces software lifecycle maintenance and requires additional open network ports.

Agentless: Agentless tools manage resources via standard protocols (SSH, WinRM, Cloud APIs), enabling rapid, low-maintenance deployments ideal for dynamic workloads.

Selecting an IaC architecture requires balancing the granular control of agent-based and imperative models against the simplicity, security, and scalability of agentless and declarative platforms.

Answer the questions below

In the scenario given, which type of IaC tool considers where you are on the map and gives instructions to reach the desired X point?

Declarative

Task 4 - IaC - The Tools Part 2

Immutable vs. Mutable Approaches: Mutable vs. immutable infrastructure defines whether environments are updated directly in place or completely replaced during a deployment.

Mutable: Mutable infrastructure updates active servers in place without provisioning new resources, but risks severe configuration drift and half-updated

states if deployment tasks fail midway.

Immutable: Immutable infrastructure provisions fresh parallel environments for updates and tears down legacy systems only after verified success, trading higher short-term resource usage for absolute environment consistency.

Immutable IaC tools: Terraform, AWS CloudFormation, Google Cloud Deployment Manager, Pulumi (some of these tools can be configured to be used with a mutable infrastructure but work better with an immutable approach)

Stateful database servers often favor a mutable update model for routine patching to avoid the complex data migration overhead and risk associated with rebuilding stateful systems from scratch.

Provisioning vs. Configuration Management: Tool classification depends on capability—distinguishing between raw resource setup/management (provisioning) and operating system or application lifecycle configuration (configuration management). Comprehensive end-to-end management spans infrastructure provisioning, ongoing infrastructure updates, initial software installation, and long-term software/configuration management.

Provisioning tools: Terraform, AWS CloudFormation, Google Cloud Deployment Manager, Pulumi

Configuration management tools: Ansible, Chef, Puppet, Saltstack

Configuration management tools like Ansible, Chef, Puppet, and SaltStack handle internal software states, allowing teams to pair a provisioner like Terraform for cloud resources with a configuration manager to automatically deploy essential security agents.

Put That All Together:

Terraform: is a declarative, agentless infrastructure provisioning tool that works with immutable infrastructure. Terraform is one of the most popular infrastructure provisioning tools out there, allowing for the management of infrastructure across multiple cloud providers like AWS, GCP and Azure.

Ansible: is a hybrid, typically agentless configuration management tool that works with mutable infrastructure. Another example of a massively popular tool in the DevSecOps space. It's slightly harder to categorise than some of the other tools, because of how this tool functions and what it does can depend on how you

employ it. For example, there is much discussion over whether Ansible is an imperative or declarative tool. In reality, Ansible is sort of a hybrid in a sense.

Pulumi: is a declarative, agentless infrastructure provisioning tool that works with immutable infrastructure. Pulumi is similar in nature to Terraform but has become increasingly popular in recent years due to its ability to let users define their desired infrastructure using familiar general-purpose languages like Python, JavaScript, Go, Java and markup languages like YAML.

Aws CloudFormation: is a declarative, agentless infrastructure provisioning tool which can be used to implement an immutable infrastructure on AWS. This is an AWS-managed service and provisions AWS cloud resources using JSON / YAML templates.

Chef: is an imperative, agent-based configuration management tool that works with mutable infrastructure. Chef gets an infrastructure to a desired state by running a series of instructions contained in a file referred to as a 'Recipe'; a collection of these files is referred to as a 'Cookbook'.

Puppet: is a declarative, agent-based configuration management tool that works with mutable infrastructure. In Puppet, you define the desired configuration state using their own domain-specific language (DSL), "Puppet Code"; Puppet then automates this configuration through a Puppet primary server and agent.

Tool	Declarative/ Imperative	Agent-Based	Type	Cloud-Agnostic
Terraform	Declarative	No	Infrastructure Provisioning	Yes
Ansible	Supports Both	No	Configuration Management	Yes
Pulumi	Declarative	No	Infrastructure Provisioning	Yes
AWS CloudFormation	Declarative	No	Infrastructure Provisioning	No (AWS only)
Google Cloud Deployment Manager	Declarative	No	Infrastructure Provisioning	No (Google Cloud only)
Chef	Imperative	Yes	Configuration Management	Yes
Puppet	Declarative	Yes	Configuration Management	Yes
SaltStack	Supports Both	Yes	Configuration Management	Yes

Static Site Scenario: The year is 2029. The destroyers have risen from the ashes of nuclear fire brought on by FlyNet, an artificial intelligence that has gained sentience. The destroyers have one objective: to destroy humankind. The resistance is led by Ron Connor, who has just received intel that the machines have discovered a technology which allows them to send a machine back to 1984 and kill his mother before he is born. Not cool! Ron Connor needs this technology to send his finest sergeant, Miles Creese, back to 1984 to protect his mother. The only problem is he needs to figure out where to find it! After a successful skirmish, Ron Connor has managed to get employee access to a FlyNet terminal. Boot up the static site, bypass the employee training and retrieve the location info he needs! What could go wrong?

Answer the questions below

| Can you retrieve the location and retrieve the flag?

thm{I4b_C0mpl3x_co0rds}

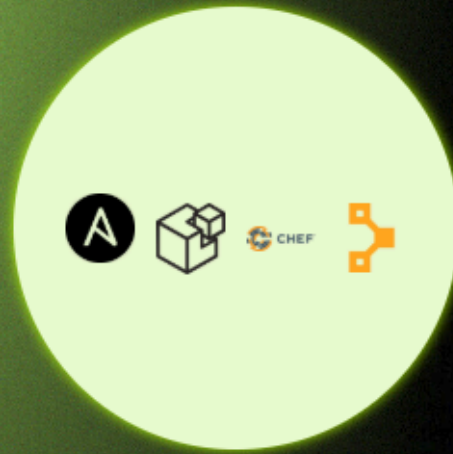
Instructions: Drag and drop the correct item into the correct box

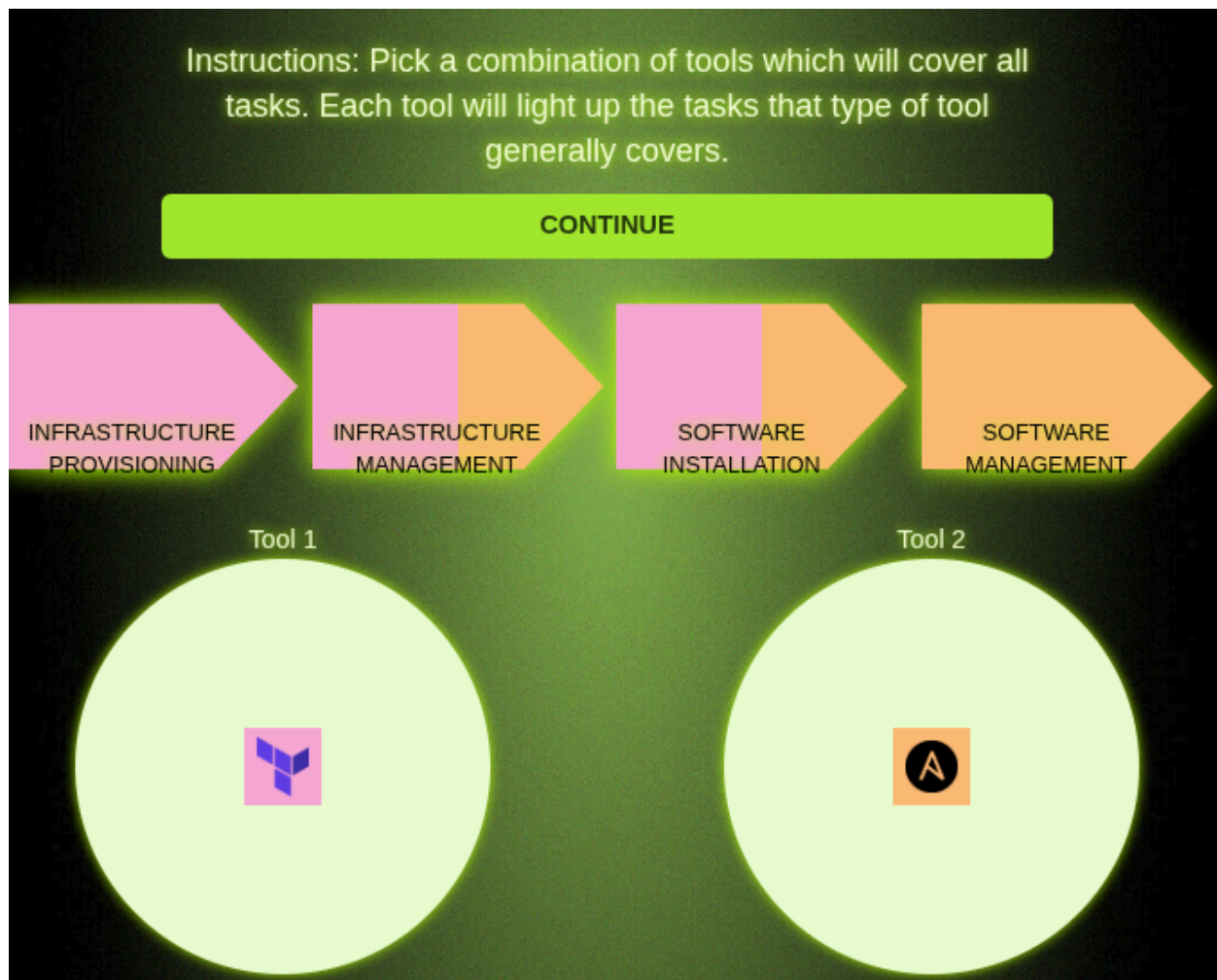
CONTINUE

PROVISIONING



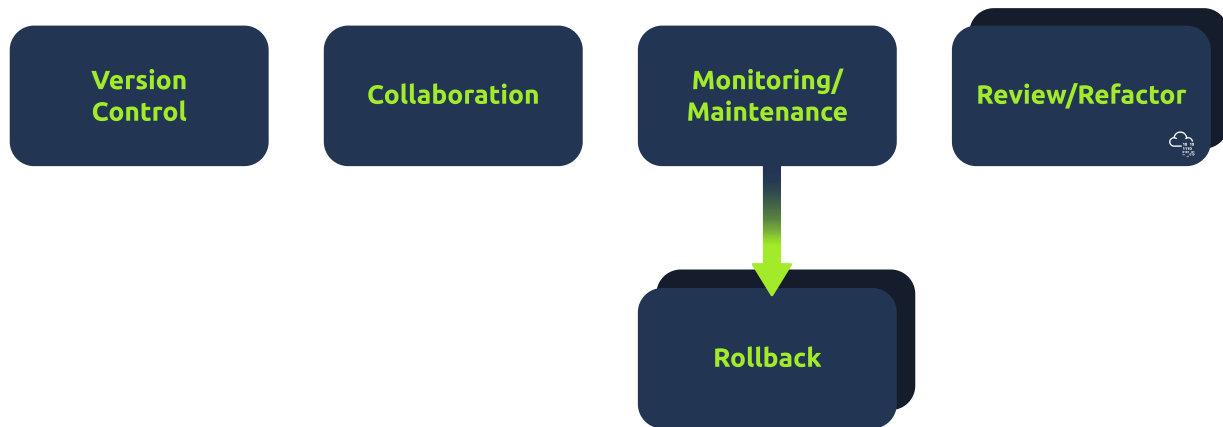
CONFIGURATION MANAGEMENT





Task 5 - Infrastructure as Code Lifecycle

The Infrastructure as Code Lifecycle (laCLC) provides a structured, non-linear framework that organizes continuous provisioning and management processes into repeatable phases. Designed for DevOps engineers, the laCLC incorporates continuous integration and continuous deployment best practices to guide both initial provisioning and long-term infrastructure support. While sharing core software engineering foundations with the SDLC—such as planning, testing, and deployment—the laCLC uses distinct phase types tailored to rapidly evolving infrastructure demands.



Breaking it Down: As mentioned above, these phases are separated into two types.

Continual (Best Practice) Phases: These phases are continually done to ensure best practices throughout the various stages of infrastructure development and management. Continual phases can trigger repeatable phases

Repeatable(Infra Creation + Config) Phases: These phases are done during the creation/configuration of an infrastructure and are done one or many times at different points and with differing variations depending on what needs to be done.

Activity: Entirely new infra



Activity: Config change



The Continual (Best Practice) Phases

Version Control	This phase involves versioning code while defining your infrastructure and making changes. This way, when it comes to the later rollback phase, there are clearly defined versions to fall back on should a new change break something.
Collaboration	When working as part of a team on infrastructure, it is crucial to communicate and collaborate so everyone is on the same page. Otherwise, this can lead to a disconnect between modular infrastructure components or general confusion over infrastructure.
Monitoring/Maintenance	Once an infrastructure has been provisioned/configured, it must be monitored for poor performance, security events, failure events, warnings, etc. Automated maintenance tasks (e.g. disk clean-up) can help avoid some of these events. In certain events, this phase can trigger the rollback phase.
Rollback	In the event of a failure event post-deployment, the infrastructure will need to be rolled back to the last known working version.
Review + Change	Just because an infrastructure works doesn't mean it's as efficient as it could be. Perhaps a new security vulnerability has been discovered, or maybe there's a new business requirement that would benefit from an infrastructure change. Infrastructure should routinely be reviewed to determine if this is the case, and changed if so.

The Repeatable (Infra Creation + Config) Phases

Design	The infrastructure needs to be designed based on needs, taking security into account throughout the whole process (for example, poor scaling policy design can lead to availability issues, which is a huge security concern).
Define	Use the design to define the infrastructure that needs to be provisioned in code.
Test	Using a linter, validate the code for syntax errors or logical issues. Before provisioning is done in

	production, test the infrastructure in a replica environment (staging).
Provision	Provision the infrastructure that has been defined in the production environment using a provisioning tool, e.g. Terraform.
Configure	Configure the provisioned environment using a configuration management tool, e.g. Ansible.

Answer the questions below

A DevSecOps Engineer at CyberMyne is looking for guidance on developing their next infrastructure. What type of phases provide guidance during the development or configuration of an infrastructure?

Repeatable

What type of phases ensure best practices throughout infrastructure development and management?

Continual

The 'Monitoring/Maintenance' continual phase can trigger which other continual phase?

Rollback

Task 6 - Virtualisation & IaC

Virtualization abstraction provides the programmatically manageable resources—such as compute, storage, and networks—that IaC tools target to spin up or teardown environments on demand.

Virtualisation Levels:

- **Hypervisor-Level Virtualization:** Hypervisors abstract physical hardware to run multiple isolated guest operating systems on a single server, delivering high isolation at the cost of slower provisioning times.
- **Container-Level Virtualization:** Containerization virtualizes at the OS kernel level, providing lightweight, rapidly deployable environments ideal for instant

scaling.

Use of Virtualisation in IaC:

Scalability	IaC uses virtualisation to meet demands (higher or lower levels of demand). If there is a sudden increase in demand (scaling policies can be defined in your code to specify when this would be required) and you need more servers, lab machines can be created to handle this increased load.
Resource Isolation	Virtualisation enabled resource isolation between different VMs/containers. Using IaC, you can define what components get how much of the available resources. This means if one component starts getting hammered (consuming all of its allocated CPU, memory, bandwidth, etc.), it won't affect the performance of the other components.
Testing / Snapshots / Rollbacks	The virtualisation of OSs, software configurations and dependencies makes it easy to package them and, using infrastructure as code, deploy them in another identical environment for testing or development. The consistency this provides, combined with the ability to take snapshots of VMs/containers at specific points in time so rollbacks can be done if a deployment goes wrong, streamlines development and testing processes.
Templates	When using virtualisation, you can create a template (or blueprint) for VMs/containers. IaC can use these templates to provision new instances. These templates cut manual provisioning and configuration time down while ensuring consistency across instances.
Multi-tenancy	Virtualisation can support multi-tenancy, meaning multiple customers/tenants can be hosted on the same physical server whilst having their resources isolated. This removes the need for dedicated servers/single-tenant environments (although sometimes they are required), and using IaC means the resources for multiple tenants can be managed using the same underlying infrastructure.
Portability	Virtualisation makes it easier to move infrastructure between cloud providers or data centres. Having this portability, as discussed, can streamline critical business processes like disaster recovery but also allows the flexibility for hybrid

cloud organisations to recreate environments regardless of their CSP.

Virtualisation Technology + IaC:

- **Orchestration Synergy:** Combining IaC with container orchestration tools like Kubernetes enables automated provisioning of underlying cluster nodes alongside the declarative deployment, scaling, and lifecycle management of containerized workloads.
- **Terraform + Kubernetes Integration:** Using Terraform modules to define Kubernetes clusters, services, and workloads standardizes deployments across environments while leveraging dependency awareness to halt downstream provisioning if cluster components fail.
- **Ecosystem Ecosystem Breadth:** Virtualization-driven IaC extends beyond cloud-managed Kubernetes to local testing environments via Vagrant and private cloud automation through platforms like OpenStack.

Answer the questions below

CyberMine is deploying the latest machine model E-1000. This model requires virtualisation at an operating system level to allow for lightweight and rapid deployment behind the scenes! What level of virtualisation would be needed for this?

Containerisation

CyberMine's E-100 Model is still very popular for all your extermination needs, this model requires multiple OS to run on a single machine. Which level of virtualisation would be needed for this?

Hypervisor

The new E-1000 model has a feature that allows it to pass through physical objects. Wild! This new feature, however, is very resource-intensive. Which 'Use of IaC' will ensure that this resource consumption won't affect the performance of the machine's other components?

Resource Isolation

Due to the resource consumption of this new feature, it requires rapid scaling of resources. Which container orchestration software can be used to automate this process?

Kubernetes

Task 7 - On-Prem IaC vs. Cloud-Based IaC

Infrastructure as code has two use cases. On-premises IaC and cloud-based IaC. Going forward, this module will go in-depth and give practical examples of both of these.

- **On-Premises Infrastructure:** On-premises IaC manages physical hardware, network devices, and local servers—often incurring higher CapEx and slower, manual scaling processes—making it best suited for legacy environments or strict data control needs.
- **Cloud-Based Infrastructure:** Cloud-based IaC leverages API-driven, elastic resources from cloud providers using tools like Terraform, CloudFormation, or ARM templates, enabling automated auto-scaling and a pay-as-you-go OpEx cost model ideal for fluctuating workloads.

Answer the questions below

Cloud-based resources are provisioned/configured in a cloud environment. Who handles the underlying infrastructure?

cloud service provider

What category does on-prem infrastructure struggle with due to hardware limitations when facing increased traffic?

scalability

Task 8 - IaC - The Final Push

The technology required to send Sergeant Miles Creese back to 1984 is being held in a lab complex. All that's left now is to capture it and retrieve the tech. There's only one issue: it's located in a heavily fortified exterminator base, and the resistance doesn't have the resources to take them on. Lucky for the resistance, though, one piece of technology they *do* have their hands on is their

'Infrastructure at a Click' weapons system. This allows resistance fighters to deploy a weapon infrastructure (requiring a network and server components so the weapons can communicate with each other) to take on the machine army! After all, weapons can be reprovisioned; resistance fighters cannot! Simply define how many weapons you want using provision points and upgrade those weapons using configure points to defeat the incoming machines! Just don't let the machines take down your server, or it's game over!

Answer the questions below

| Can you get the flag using your infrastructure as code skills?

thm{1Nfr4StrUctUr3_Pr0}

Points Allocation

You have been assigned 2 points, allocate them wisely.
Remaining Points: 0/2

Provisioning ⓘ

▼

1

▲

Configuration ⓘ

▼

1

▲

Next

Select Infrastructure

Each Infrastructure has its own perks.

☒ On-premises ⓘ

☐ Cloud-based ⓘ

Back

Save